

Lecture 21: GPT Architecture

Generative Pre-Training for Language

Models of Language and Conversation

Week 7

Today's Journey

1. **The GPT Revolution** - Pre-training + fine-tuning paradigm
2. **Transformer Decoder** - Architecture walkthrough
3. **Masked Self-Attention** - The core innovation
4. **Two-Stage Training** - Pre-training and fine-tuning
5. **Hands-on Code** - Building GPT blocks in PyTorch
6. **Key Results** - What made GPT successful

The GPT Revolution

Discussion

What made GPT different from previous language models?

Key Innovation: Pre-training + Fine-tuning

- **Pre-train** on massive unlabeled text (unsupervised)
- **Fine-tune** on specific tasks (supervised)
- Transfer learning for NLP
- One model, many tasks

Paradigm Shift

Before GPT: Task-specific models trained from scratch

After GPT: General-purpose models adapted to tasks

From BERT to GPT: Different Approaches

BERT (2018):

- **Masked Language Model**
- Bidirectional context
- Fill in the blank
- Great for understanding
- Not designed for generation

Example:

"The [MASK] sat on the mat"

GPT (2018):

- **Autoregressive Model**

Left-to-right (causal)

The Transformer Decoder

GPT uses the Transformer *decoder* architecture

Data Flow:

1. Input tokens
2. Token + Position embeddings
3. N x Transformer blocks:
 - Masked self-attention
 - Feed-forward network
4. Output logits

Key Components:

- **Embeddings:** Map tokens to vectors

Masked Attention: Only see past tokens

Transformer Decoder: Worked Example

Example: Processing "The cat sat"

1Step 1: Token Embeddings

2"The" → [0.2, -0.1, 0.8, ...] (768-dim vector)

3"cat" → [0.5, 0.3, -0.2, ...]

4"sat" → [-0.1, 0.7, 0.4, ...]

5

6Step 2: Add Position Embeddings

7Position 0 embedding + "The" embedding

8Position 1 embedding + "cat" embedding

9Position 2 embedding + "sat" embedding

10

11Step 3: Masked Self-Attention (for each layer)

12"The" attends to: [The]

13"cat" attends to: [The, cat]

14"sat" attends to: [The, cat, sat]

15

16Step 4: Predict Next Token

Masked Self-Attention

The Core Innovation: Causal Masking

Attention Matrix for "The cat sat on":

	The	cat	sat	on
The	1.0	-inf	-inf	-inf
cat	0.3	0.7	-inf	-inf
sat	0.2	0.3	0.5	-inf
on	0.1	0.2	0.3	0.4

Values show attention weights after softmax (masking sets values to -inf)

Each token can only attend to itself and previous tokens

Causal Mask: Code Implementation

```
1import torch
2
3def create_causal_mask(seq_len):
4    """Create lower-triangular mask for causal attention."""
5    # Create a matrix of ones
6    mask = torch.ones(seq_len, seq_len)
7    # Keep only lower triangle (including diagonal)
8    mask = torch.tril(mask)
9    return mask
10
11# Example for sequence length 4
12mask = create_causal_mask(4)
13print(mask)
14# tensor([[1., 0., 0., 0.],
15#         [1., 1., 0., 0.],
16#         [1., 1., 1., 0.],
17#         [1., 1., 1., 1.]])
18
```

Position Encoding in GPT

Why we need position information:

The Problem

Self-attention is *permutation invariant* - it doesn't inherently know word order!

GPT's Solution: Learned Position Embeddings

- Each position gets a learnable embedding
- Added to token embeddings: `embedding = token_emb + pos_emb`
- Model learns position patterns during training
- Maximum sequence length: 512 tokens (GPT-1)

Alternative: Sinusoidal

(Used in original Transformer)

Position Embedding: Code Example

```
1import torch
2import torch.nn as nn
3
4class GPTEmbedding(nn.Module):
5    def __init__(self, vocab_size, d_model, max_seq_len):
6        super().__init__()
7        # Token embeddings: word -> vector
8        self.token_embed = nn.Embedding(vocab_size, d_model)
9        # Position embeddings: position -> vector
10       self.pos_embed = nn.Embedding(max_seq_len, d_model)
11
12    def forward(self, x):
13        seq_len = x.size(1)
14        # Create position indices [0, 1, 2, ..., seq_len-1]
15        positions = torch.arange(seq_len, device=x.device)
16        # Combine: token embedding + position embedding
17        return self.token_embed(x) + self.pos_embed(positions)
18
```

Position Embedding: Code Example

```
21tokens = torch.tensor([[101, 2054, 2003]]) # "The cat sat"  
22embeddings = embed(tokens) # Shape: (1, 3, 768)
```

...continued

GPT-1 Model Specifications

Component	Value
Layers	12
Hidden size	768
Attention heads	12
Max sequence length	512 tokens
Vocabulary size	40,000 (BPE)
Training data	BooksCorpus (7,000 books)
Training tokens	~5 billion
Total parameters	~117 million

GPT Architecture in Code

```
1import torch.nn as nn
2
3class GPTBlock(nn.Module):
4    def __init__(self, d_model, n_heads):
5        super().__init__()
6        self.attention = MaskedMultiHeadAttention(d_model, n_heads)
7        self.norm1 = nn.LayerNorm(d_model)
8        self.ffn = nn.Sequential(
9            nn.Linear(d_model, 4 * d_model),
10            nn.GELU(), # GPT uses GELU, not ReLU
11            nn.Linear(4 * d_model, d_model)
12        )
13        self.norm2 = nn.LayerNorm(d_model)
14
15    def forward(self, x):
16        # Pre-norm architecture (LayerNorm before sublayer)
17        # Self-attention with residual connection
18        x = x + self.attention(self.norm1(x))
```

GPT Architecture in Code

```
21         return x
```

...continued

Complete GPT Model Structure

```
1 class GPT(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, n_heads, max_len):
3         super().__init__()
4         self.token_embed = nn.Embedding(vocab_size, d_model)
5         self.pos_embed = nn.Embedding(max_len, d_model)
6         self.blocks = nn.ModuleList([
7             GPTBlock(d_model, n_heads) for _ in range(n_layers)
8         ])
9         self.norm = nn.LayerNorm(d_model)
10        self.head = nn.Linear(d_model, vocab_size)
11
12    def forward(self, x):
13        seq_len = x.size(1)
14        positions = torch.arange(seq_len, device=x.device)
15        x = self.token_embed(x) + self.pos_embed(positions)
16        for block in self.blocks:
17            x = block(x)
18        x = self.norm(x)
```

Two-Stage Training

Stage 1: Pre-training

- Dataset: BooksCorpus (5B tokens)
- Objective: Next token prediction
- Duration: Weeks of training
- Result: General language model

Stage 2: Fine-tuning

- Dataset: Task-specific (1K-100K examples)
- Objective: Task loss + LM loss
- Duration: Hours of training
- Result: Task-specialized model

Pre-training: Language Modeling

Objective: Predict next token

Example Training Sequence

Text: "The quick brown fox jumps over the lazy dog"

Training examples (teacher forcing):

- Input: "The" -> Target: "quick"
- Input: "The quick" -> Target: "brown"
- Input: "The quick brown" -> Target: "fox"
- Input: "The quick brown fox" -> Target: "jumps"
- ...and so on

Self-supervised learning:

Pre-training: Worked Example

How loss is computed for one sequence:

```
1# Input sequence: "The cat sat on the"
2tokens = [101, 2054, 2003, 2006, 1996] # Token IDs
3
4# Model predicts probability distribution for each position
5# Position 0: P(next | "The") = {"cat": 0.3, "dog": 0.2, ...}
6# Position 1: P(next | "The cat") = {"sat": 0.4, "ran": 0.1, ...}
7# etc.
8
9# Target tokens (shifted by 1)
10targets = [2054, 2003, 2006, 1996, 2282] # "cat sat on the mat"
11
12# Cross-entropy loss at each position
13loss_0 = -log(0.3) # P("cat" | "The")
14loss_1 = -log(0.4) # P("sat" | "The cat")
15loss_2 = -log(0.35) # P("on" | "The cat sat")
16# ...
```

Fine-tuning for Downstream Tasks

Task Format: Input + Delimiter + Label

Text Classification:

```
1[START] This movie is amazing! [DELIM]
```

Model predicts: "Positive"

Entailment:

```
1[START] Premise [DELIM] Hypothesis [DELIM]
```

Model predicts: "Entailment" / "Contradiction"

Question Answering:

Fine-tuning: Concrete Example

Sentiment Classification Example:

```
1# Training example
2text = "This movie was absolutely fantastic!"
3label = "positive"
4
5# Format for GPT
6input_text = "[START] This movie was absolutely fantastic! [DELIM]"
7# Tokenize: [50256, 1212, 3807, 373, 5765, 12779, 0, 50257]
8
9# During fine-tuning:
10# 1. GPT processes the input
11# 2. Take the hidden state at [DELIM] position
12# 3. Pass through classification head
13logits = classification_head(hidden_state) # [pos_score, neg_score]
14# 4. Compute cross-entropy with label
15loss = cross_entropy(logits, label_id)
16
```

Fine-tuning Implementation Details

What changes during fine-tuning?

1. Add task-specific input transformations

- Format inputs with delimiters
- Add special tokens

2. Add classification head (optional)

- Linear layer for classification
- Or use language modeling head

3. Train with supervised objective

- Cross-entropy loss
- Much lower learning rate

GPT-1 Results

Performance on GLUE Benchmark:

Task	Previous SOTA	GPT-1
Question Answering	86.7	88.1
Semantic Similarity	85.0	85.8
Text Classification	93.0	94.2
Natural Language Inference	80.6	82.1

Key Findings

- GPT achieved SOTA on 9 out of 12 tasks studied
- Large improvements on tasks with less training data
- Transfer learning works for NLP!

Zero-Shot and Few-Shot Learning

Discussion

What if we could use the model *without* fine-tuning?

Definitions:

- **Zero-shot:** No task-specific training examples
- **Few-shot:** A few examples (1-10) as context
- **Fine-tuning:** Full supervised training

GPT-1 observations:

- Fine-tuning works best
- Zero-shot performance is weak
- Model has learned a lot, but needs task formatting

Zero-Shot vs Fine-tuning: Example

Task: Sentiment Classification

Zero-shot (GPT-1):

```
1Input: "Review: Great movie!  
2Sentiment:"  
3Output: ??? (unreliable)
```

GPT-1 might just continue the text randomly.

After Fine-tuning:

```
1Input: "[START] Great movie! [DELIM]"  
2Output: "Positive" (reliable)
```

Visualizing What GPT Learns

Layer-by-layer analysis of GPT representations:

Layer	What it learns	Example
1-3	Word order, grammar	"The cat" vs "cat The"
4-6	Word meanings, syntax	Subject-verb agreement
7-9	Semantic relationships	"bank" in different contexts
10-12	World knowledge, task	"Paris is the capital of..."

Layer analysis shows:

- Early layers: Syntactic patterns
- Middle layers: Semantic understanding
- Later layers: Task-specific reasoning

The Generative Pre-training Paradigm

Why "Generative Pre-Training" was revolutionary:

1. Unified Architecture

- Same model for all tasks
- vs. task-specific architectures

2. Transfer Learning

- Learn once, apply many times
- vs. training from scratch

3. Scalability

- More data -> Better performance
- Clear path to improvement

Limitations of GPT-1

What GPT-1 couldn't do well:

- **Zero-shot performance:** Needed fine-tuning for each task
- **Model size:** 117M parameters (small by today's standards)
- **Training data:** Limited to ~5B tokens
- **Context length:** Only 512 tokens
- **Generation quality:** Sometimes incoherent
- **Factual accuracy:** Prone to hallucinations

Discussion

How would you address these limitations?

(Spoiler: GPT-2 and GPT-3 tried to solve these!)

Comparison with BERT

Aspect	BERT	GPT
Training objective	Masked LM	Autoregressive LM
Context	Bidirectional	Left-to-right
Best for	Understanding	Generation
Fine-tuning	Task-specific heads	Unified format
Parameters (base)	110M	117M

Think about it!

BERT dominated understanding tasks (2018-2019), but GPT's approach proved more scalable and versatile. Why?

Key Takeaways

1. GPT introduced generative pre-training

- Transformer decoder architecture
- Autoregressive language modeling

2. Two-stage training paradigm

- Unsupervised pre-training on massive text
- Supervised fine-tuning on task data

3. Masked self-attention is key

- Enables autoregressive generation
- Prevents information leakage

4. Transfer learning for NLP

Readings

Required Reading

Radford et al. (2018) - "Improving Language Understanding by Generative Pre-Training"

[\[PDF\]](#)

Recommended Readings

- **Vaswani et al. (2017)** - "Attention is All You Need" [\[ArXiv\]](#)
- **Devlin et al. (2018)** - "BERT: Pre-training of Deep Bidirectional Transformers" [\[ArXiv\]](#)
- **The Illustrated GPT-2** by Jay Alammar [\[Blog\]](#)

Next Lecture Preview

Lecture 22: Scaling Up to GPT-3 and Beyond

- GPT-2: Language models as unsupervised multitask learners
- GPT-3: Few-shot learning at scale
- Scaling laws: Why bigger is better
- Emergent abilities of large language models
- From GPT-3 to ChatGPT

Questions?